

Lab Report Structure and Content Requirements

Each lab report should follow a consistent structure. For the final project submission, a comprehensive report covering all 8 labs is typically required.

I. General Report Structure

Section	Purpose	Typical Length
1. Cover Page	Project Title (HRM System), Course Info (Software Architecture - N01), Team Members (Group 05), Date.	1 Page
2. Abstract/Summary	Brief overview of the project (HRM System) and the key architectural patterns implemented and compared (Microservices, Layered, Event-Driven).	1 Paragraph
3. Lab Specific Section (I-VIII)	Detailed documentation of the activities, diagrams (UML, C4), code snippets, and test results for each lab.	Variable
4. Conclusion & Reflection	Summary of architectural knowledge gained and key trade-offs observed in the actual project.	1 Page

II. Required Content Breakdown by Lab

The following outlines the specific deliverables required in the content section for each of the eight labs, tailored specifically to your HRM project's tech stack (Node.js, PostgreSQL, Redis).

Lab 1: Requirements Elicitation & Modeling

- **Requirements Tables:** Documented Functional (Attendance, Leave, Payroll), Non-Functional, and the three Architecturally Significant Requirements (ASRs - e.g., handling concurrent payroll calculations for thousands of employees, ensuring Database High Availability).
- **Modeling Artifact:** UML Use Case Diagram showing the system boundary, Actors (STAFF, MANAGER, SUPERADMIN), core Use Cases (e.g., Submit Leave Request), and the use of <<include>> and <<extend>> relationships for a critical path (e.g., Attendance flow including Late/Early calculation logic).

Lab 2: Layered Architecture Design (Logical View)

- **Documentation:** Table defining the four layers inside a single service (e.g., leave_service): Presentation (Router), Business Logic (Service), Persistence (Database Queries), Data (PostgreSQL), and their strict downward dependencies.
- **Modeling Artifact:** UML Component Diagram showing the logical components (LeaveRouter, LeaveService, LeaveRepository) within their respective layers, explicitly illustrating the interfaces (lollipop/socket notation) and dependency flows.

Lab 3: Layered Architecture Implementation (CRUD)

- **Code Structure:** Screenshot or listing of a standard service directory structure demonstrating separation of concerns (e.g., routes/, controllers/, services/, models/).
- **Code Snippets:** Node.js/Express code snippets demonstrating the flow: Router receiving requests and delegating to Service, Service containing business rules (e.g., Max 3 consecutive leave days), and Database Queries (using the pg library) handling data access.
- **Verification:** Screenshots (from Postman or React UI) showing successful CRUD operation tests (e.g., successful POST request to create a leave ticket, GET request to fetch work schedules).

Lab 4: Microservices Decomposition & Communication

- **Decomposition Table:** Table listing the 5 core Microservices (auth_service, leave_service, salary_service, attendance_service, noti_service), their business capabilities, and data ownership.
- **Documentation:** Defined Service Contract (API Endpoints) for the Leave Service (e.g., GET /api/leaves, POST /api/leaves, GET /api/leaves/schedule/weekly).
- **Modeling Artifact:** C4 Model (Level 1: System Context Diagram) showing the HRM system boundary, external Actors, External Systems (NodeMailer Email Service), and high-level communication types.

Lab 5: Implementing a Microservice

- **Setup:** List of installed npm dependencies (Express, pg, jsonwebtoken). Note the configuration to run on an independent port and connect to the internal PostgreSQL DB.

- **Code Snippets:** Schema structure (from [init.sql](#)) and Express routes for the core functionalities.
- **Verification:** Terminal log screenshots showing the service running independently and Postman/cURL test results confirming the API works perfectly when detached from the main system.

Lab 6: Introducing the API Gateway Pattern

- **Code Snippet:** The [docker-compose.yml](#) configuration defining the Nginx and Kong API Gateway services.
- **Code Snippet:** The [nginx/nginx.conf](#) file demonstrating the request forwarding logic using proxy_pass http://kong_upstream; and Frontend/Backend routing rules.
- **Verification:** Test output showing the following three critical results:
 1. 200 OK response via Nginx port 80.
 2. 401/403 response if no valid Token is provided (handled at the Gateway/Middleware level).
 3. 502 Bad Gateway / 503 response demonstrating Nginx's graceful handling when a backend container (like leave_service) is abruptly stopped.

Lab 7: Event-Driven Architecture (EDA) & Integration

- **Setup:** [docker-compose.yml](#) configuration showing the successful deployment of the Message Broker (Redis container on port 6379).
- **Code Snippets (Producer):** Backend logic (e.g., in leave_service) utilizing socket.io with redis-adapter or directly Publishing an event (e.g., new_leave_request) to Redis when an employee submits a form.
- **Code Snippets (Consumer):** The listener service (Consumer) logic handling the task of sending Toast notifications to the Admin/Manager without blocking the main request creation flow.
- **Verification:** Evidence (Terminal logs or UI Video) proving asynchronous decoupling: The employee submits a request and immediately gets a "Success" response, while the notification/email system processes in the background without slowing down the UX.

Lab 8: Deployment View & Quality Attribute Analysis (ATAM)

- **Modeling Artifact:** UML Deployment Diagram based on [docker-compose.yml](#). Showing Nodes (Nginx, Application Cluster), Artifacts (Node.js Microservices,

Kong Gateway), and specifically the High Availability Database clusters (Postgres Spilo/Patroni, MongoDB Replica Set, Redis).

- **Analysis:** Completed ATAM Analysis Table evaluating the Monolithic vs. Microservices architecture against two real HRM scenarios:
 - Scalability (SS1): How the system handles 5000 employees checking their payroll simultaneously on payday.
 - Availability (AS1): How the system reacts if noti_service crashes or one Database node (postgres-1) goes down.
- **Reflection:** A concise paragraph identifying the main Architectural Trade-offs learned from the project (e.g., achieving excellent scalability and fault isolation at the cost of immense complexity in DevOps deployment and distributed data tracing).